Índice

1. Propiedades de la suma y del producto por un escalar en $\mathbb{R}^n$ **2**
1.1. Vídeo completo - Propiedades de la suma y del producto por un escalar 2
1.1.1. Escena 1 - Introducción 2
1.2. Escena 1 - Introducción 4

2. Trozos comunes de código **4**
2.1. Carga de las librerías Manim, manim_voiceover y NacAL 4
2.2. Servicios de voz 4

Lección 1 del curso - Vídeo 5

Marcos Bujosa

12 de abril de 2026

1. Propiedades de la suma y del producto por un escalar en $\mathbb{R}^n$

1.1. Vídeo completo - Propiedades de la suma y del producto por un escalar

```
<<Carga de la librería Manim y NacAL>>
<<Definición de mi escena con voz>>
<<Servicio de voz AzureService>>
<<Servicio de voz de Google>>
<<Definición de escena final con copyright>>

class L01_V05_PropiedadesDeLasOperacionesConVectores(MiEscenaConVoz):
    def construct(self):
        <<Azure Voice>>

        <<TexTemplate con LaTeX-nacal>>

        <<Primera escena>>

        self.play(*[FadeOut(mob)for mob in self.mobjects] )  # borrado de todo

        <<Creditos finales>>
```

1.1.1. Escena 1 - Introducción

Primera escena

```
self.creditos()

# Portada
titulo = Title("Propiedades de las operaciones con vectores de  " + r"\R[n]",
               tex_template = myTemplate,
               font_size=50).set_color(BLUE)
self.play(Write(titulo))
self.pausa_media()
# self.play(FadeOut(titulo))

igual = MathTex(r"=",
                tex_template = myTemplate,)

mas = MathTex(r"+",
              tex_template = myTemplate,)

va = MathTex(r"\Vect{a}",
             tex_template = myTemplate)

vb = MathTex(r"\Vect{b}",
             tex_template = myTemplate)

# suma
cvab = MathTex(r"\elemRp{\Vect{a}+\Vect{b}}{i}",
               tex_template = myTemplate)
```

```
cva  = MathTex(r"\eleVR{a}{i}",
               tex_template = myTemplate)

cvb  = MathTex(r"\eleVR{b}{i}",
               tex_template = myTemplate)

eq_suma = VGroup(cvab.copy(),
                 igual.copy(),
                 cva.copy(),
                 mas.copy(),
                 cvb.copy()).arrange(RIGHT).scale(1.5)

# producto
cvla = MathTex(r"\elemRp{\lambda\Vect{a}}{i}",
               tex_template = myTemplate)

lca  = MathTex(r"\lambda\eleVRpE{a}{i}",
               tex_template = myTemplate)
eq_producto = VGroup(cvla.copy(),
                     igual.copy(),
                     lca.copy()).arrange(RIGHT).scale(1.5).next_to(eq_suma,
                                                                   DOWN,
                                                                   aligned_edge=LEFT,
                                                                   buff=1.2)

with self.voiceover(text="""En los vídeos anteriores hemos visto dos operaciones con vectores de Rn:""") as tracker:
    self.pausa_corta(tracker.duration)

with self.voiceover(text="""La suma,""") as tracker:

    self.play(FadeIn(eq_suma[0][0][1:4],
                     rate_func=rate_functions.exponential_decay),
              run_time=2*tracker.duration/3)

with self.voiceover(text="""y el reescalado""") as tracker:

    self.play(FadeIn(eq_producto[0][0][1:3],
                     rate_func=rate_functions.exponential_decay),
              run_time=2*tracker.duration/3)

with self.voiceover(text="""Ambas operaciones se definen componente a componente.""") as tracker:

    self.play(FadeIn(eq_suma[0][0][0],  # paréntesis inicial
                     eq_suma[0][0][4:], # cierre de paréntesis y selector
                     eq_producto[0][0][0],  # paréntesis inicial
                     eq_producto[0][0][3:], # cierre de paréntesis y selector
                     scale=.5,
                     rate_func=rate_functions.exponential_decay),
              run_time=tracker.duration)

with self.voiceover(text="""Las componentes del 'vector suma' son la suma de las componentes,""") as tracker:

    self.play(FadeIn(eq_suma[1:],       # resto de la ecuación
                     scale=.1,
                     rate_func=rate_functions.exponential_decay),
              run_time=3*tracker.duration/5)
    self.play(Indicate(eq_suma[2][0]),
              Indicate(eq_suma[4][0]))

with self.voiceover(text="""y las componentes del 'vector reescalado' son las componentes reescaladas.""") as tracker:

    self.play(FadeIn(eq_producto[1:],       # resto de la ecuación
                     scale=.1,
                     rate_func=rate_functions.exponential_decay),
              run_time=3*tracker.duration/5)
    self.play(Indicate(eq_producto[2][0][0]),
              Indicate(eq_producto[2][0][2:5]))
```

```
with self.voiceover(text="""Aunque vimos la interpretación geométrica, en R2 y en R3, de ambas operaciones,
          señalamos que lo importante no es la interpretación geométrica.
          Lo más importante es que son operaciones definidas componente a componente
          que se traducen en reglas simples de manipulación simbólica.
          En este caso se traducen en operaciones entre números reales;
          sumas de números en el primer caso y productos en el segundo.""") as tracker:

    self.pausa_corta(tracker.duration)
```

1.2. Escena 1 - Introducción

```
class L01_V05_E01_Introduccion(MiEscenaConVoz):
    def construct(self):
        <<Gogle Voice>>

        <<TexTemplate con LaTeX-nacal>>

        <<Primera escena>>
```

2. Trozos comunes de código

Azure Voice

```
self.set_speech_service(AzureService(voice="es-ES-AlvaroNeural")) #self.set_speech_service(AzureService(voice="en-GB-RyanNeural"

def slow(text, rate="-10%"):
    return f'<speak><prosody rate="{rate}">{text}</prosody></speak>'
```

Gogle Voice

```
self.set_speech_service(GTTSService(lang="es", tld="com"))
```

TexTemplate con LaTeX-nacal

```
myTemplate = TexTemplate()
myTemplate.add_to_preamble(r"""\usepackage{nacal} """)
```

2.1. Carga de las librerías Manim, manim_voiceover y NacAL

Carga de la librería Manim y NacAL

```
from manim import *
from manim_voiceover import VoiceoverScene
import nacal as nc
import sympy as sp
```

2.2. Servicios de voz

Servicio de voz AzureService

```
from manim_voiceover.services.azure import AzureService
```

Servicio de voz de Google

```
from manim_voiceover.services.gtts import GTTSService
```

Definición de algunas subclases con definiciones generales

Definición de mi escena con voz

```
class MiEscenaConVoz(VoiceoverScene):
    def pausa_muy_corta(self, n=0.3):
        self.wait(n)
    def pausa_corta(self, n=0.5):
        self.wait(n)
    def pausa(self, n=1):
        self.wait(n)
    def pausa_media(self, n=1.5):
        self.wait(n)
    def pausa_larga(self, n=3):
        self.wait(n)
    def pausa_muy_larga(self, n=5):
        self.wait(n)

    <<Créditos en distintas partes de la pantalla>>
```

Créditos en distintas partes de la pantalla

```
def creditos(self, variante=1):
    def analisis_opcion_elegida(tipo):
        'Análisis de las opciones de eliminación elegidas'
        lista = [100,20,10,4,2,1]
        opcion = set()
        for t in lista:
            if (tipo - (tipo % t)) in lista:
                opcion.add(tipo - (tipo % t))
                tipo = tipo % t
        return opcion

    copyright = Tex(r"\textcopyright{\;} 2026\; Marcos Bujosa  ")

    if 1 in analisis_opcion_elegida(variante):
        stampDcha  = VGroup(copyright.copy()).rotate( PI/2).scale(0.5).to_edge(RIGHT, buff=0.1).set_color(GRAY_D)
        self.add(stampDcha)
    if 2 in analisis_opcion_elegida(variante):
        stampIzda  = VGroup(copyright.copy()).rotate(-PI/2).scale(0.5).to_edge(LEFT, buff=0.1).set_color(GRAY_D)
        self.add(stampIzda)
    if 4 in analisis_opcion_elegida(variante):
        stampBottom= VGroup(copyright.copy()).rotate(    0).scale(0.5).to_edge(DOWN, buff=0.1).set_color(GRAY_D)
        self.add(stampBottom)
    if 10 in analisis_opcion_elegida(variante):
        stampTop   = VGroup(copyright.copy()).rotate(    0).scale(0.5).to_edge(  UP, buff=0.1).set_color(GRAY_D)
        self.add(stampTop)
```

Definición de escena final con copyright

```
class ZCreditosFinales(Scene):
    def construct(self):
        copyright = Tex(r"\textcopyright{\;} 2026 \; Marcos Bujosa")
        github = Tex(r"\texttt{https://github.com/mbujosab}").next_to(copyright, DOWN)
        CGG  = VGroup(copyright,github).scale(1.1)
        self.add(CGG)
        self.wait(10)
```